Universidad Distrital Francisco José de Caldas

Faculty of Engineering
Systems Engineering Program

Project Management and Enhanced Design Document

Workshop 3: Robust Design, Quality, Risk, and Operational Planning

Authors (Engineering Team):
Omar Yesid Fonseca López (oyfonsecal@udistrital.edu.co)
Daniel Felipe Barrera Suárez (dfbarreras@udistrital.edu.co)
Daniel Mateo Ballesteros Molina (dmballesterosm@udistrital.edu.co)
Dania Lizeth Guzmán Triviño (dlguzmant@udistrital.edu.co)

Course: Systems Analysis & Design
Professor: Eng. Carlos Andrés Sierra, M.Sc.

Bogotá, Colombia
2026

Abstract

This document presents the consolidation of **Workshop 3** for the **Smart Campus Energy Management System** project at Sabio Caldas Campus. Building upon the systems analysis conducted in Workshop 1 and the preliminary design developed in Workshop 2, this document formulates a robust design proposal focused on technical feasibility, software quality, risk mitigation, and preparation for a controlled pilot implementation. The system adopts a distributed user-space architecture with autonomous agents capable of collecting telemetry, detecting inactivity, executing energy-saving policies, and operating with fault tolerance through local persistence and deferred synchronization.

In addition to architectural strengthening, this document incorporates a project management structure including a project charter, role definitions, schedule, resource plan, communication and control plan, implementation strategy, training plan, and continuous improvement framework. All of this demonstrates that the solution not only addresses the identified problem of energy waste in university laboratories but is also sufficiently structured to advance toward a measurable, controlled Proof of Concept aligned with good engineering and management practices.

The complete project repository is available at: https://github.com/yecop/Systems-Analysis-and-Design

Contents

1 Executive Summary **1**

2 Project Charter **1**
2.1 Project Purpose . . . 1
2.2 Project Scope . . . 1
2.3 Project Objectives . . . 1
2.4 Project Stakeholders . . . 2
2.5 Success Criteria . . . 2

3 System Evolution and Lessons Learned **2**
3.1 Synthesis of Workshop 1 . . . 2
3.2 Synthesis of Workshop 2 . . . 2
3.3 Improvements Introduced in Workshop 3 . . . 3
3.4 Risks Identified and Mitigated . . . 3

4 Robust System Architecture **3**
4.1 Architecture Overview . . . 3
4.2 Quality Attributes Mapping . . . 3
4.3 Integration and Interfaces . . . 3
4.4 Fault-Tolerant Architecture . . . 4

5 Risk Management Plan **4**
5.1 Risk Identification . . . 4
5.2 Risk Categories . . . 4
5.3 Risk Matrix and Mitigation Strategies . . . 5
5.4 Risk Monitoring and Control . . . 6
5.5 Contingency Plans . . . 6

6 Quality Assurance Framework **6**
6.1 Quality Objectives . . . 6
6.2 Quality Metrics . . . 7
6.3 Validation Methods . . . 7
6.4 Acceptance Criteria . . . 7
6.5 QA Flow . . . 7

7 Project Management Plan **8**
7.1 Team Structure and Roles . . . 8
7.2 Project Schedule . . . 9
7.3 Milestones, Deliverables, Dependencies, and Critical Path . . . 9
7.4 Resource Plan . . . 9
7.5 Budget and Resource Plan . . . 10
7.6 Communication and Control Plan . . . 10

8 Implementation and Readiness Strategy **10**
8.1 Deployment Strategy . . . 10
8.2 Infrastructure Requirements . . . 11
8.3 Dependencies and Preconditions . . . 11
8.4 Change Management . . . 11
8.5 Training Plan . . . 11
8.6 User Adoption Strategy . . . 12

9 Continuous Improvement Strategy **12**
9.1 Feedback Mechanisms . . . 12
9.2 Adaptive Management . . . 12
9.3 Post-Implementation Metrics . . . 12

10 Visual Representations and Tools **12**

11 References **13**

12 Appendices **14**
12.1 Repository Structure and Deliverables . . . 14
12.2 Quality Assurance Checklist . . . 14
12.3 Detailed Risk Register . . . 15
12.4 Project Repository Link . . . 15

List of Figures

1 Decentralized robust architecture with fault tolerance and local persistence 4
2 Matrix visualization of risk impact and probability 6
3 Iterative Quality Assurance process flow 8
4 Visual timeline of the Proof of Concept (6 weeks) 9
5 Quality Assurance and Testing Framework Flow 13

List of Tables

1 Project risk management matrix 5
2 Team roles and responsibilities 8
3 General project schedule 9
4 Budget estimation for the PoC execution 10
5 Pilot implementation phases 11
6 Repository structure and workshop deliverables 14
7 Detailed risk register 15

1 Executive Summary

This document synthesizes the evolution of a system aimed at reducing energy waste in the laboratories of Sabio Caldas Campus, particularly in scenarios where equipment and peripherals remain powered on despite the absence of users. In *Workshop 1*, the problem was identified from a systemic perspective, recognizing human, technical, and organizational factors. In *Workshop 2*, a software-based solution was proposed that avoided dependencies on unauthorized IoT infrastructure. In this *Workshop 3*, the proposal is strengthened through principles of robustness, quality control, risk management, and project planning.

The current proposal relies on local agents capable of monitoring activity, applying safe suspension rules, and reporting information to a database and visualization layer. Given the institutional environment's network restrictions, contingency mechanisms such as encrypted local storage and deferred synchronization are incorporated. Quality criteria, metrics, validation plans, and a pilot implementation roadmap are also defined, transforming the project from a technical idea into a more structured initiative prepared for gradual and controlled execution.

2 Project Charter

2.1 Project Purpose

The purpose of this project is to design, develop, and validate a Proof of Concept (PoC) for an intelligent energy management system that reduces unnecessary consumption in university laboratories by automating suspension actions and monitoring computer equipment activity. The system seeks to demonstrate that it is possible to improve campus energy efficiency without requiring deep changes to physical infrastructure or installation of additional network devices.

2.2 Project Scope

The scope includes robust architecture design, implementation of an autonomous Python agent, incorporation of fault tolerance mechanisms, quality testing, and a controlled pilot implementation on a reduced set of equipment. It also includes telemetry capture, local and remote event storage, and construction of an analytical dashboard to visualize preliminary energy savings results.

Large-scale institutionalization, acquisition of new physical infrastructure, and official integration with all university administrative systems are not within scope. The project is limited to an academic and technical Proof of Concept that allows feasibility and performance evaluation.

2.3 Project Objectives

- Design a robust software architecture that operates within institutional network and security restrictions.
- Implement an agent capable of detecting inactivity and executing energy-saving policies safely.

- Mitigate technical and operational risks through local persistence mechanisms, load validation, and deferred synchronization.
- Validate the solution through a controlled pilot with a set of technical and operational metrics.
- Produce evidence of feasibility for future scaling phases within the institution.

2.4 Project Stakeholders

The main project stakeholders include student users of the laboratories, faculty members responsible for the rooms, the engineering team designing and implementing the solution, technical support staff, campus administration interested in energy efficiency and security, and the course that frames the academic development of the project. Each of these actors has different interests: usability, operational continuity, energy savings, ease of maintenance, risk control, and solution quality.

2.5 Success Criteria

The project will be considered successful if the pilot demonstrates that the agent can adequately detect inactivity periods, execute energy-saving actions without affecting critical academic activities, maintain record integrity even during network failures, and produce sufficient data for impact analysis. Additionally, success will be measured by documentation clarity, design maintainability, and the possibility of future system expansion.

3 System Evolution and Lessons Learned

3.1 Synthesis of Workshop 1

In the first phase, a comprehensive analysis of the real system was conducted, identifying energy waste as a recurrent problem associated with university laboratory usage. A significant portion of consumption was observed not to be due exclusively to hardware deficiencies, but rather to usage practices such as leaving peripherals on, maintaining inactive sessions for long periods, or failing to perform timely shutdowns at the end of academic activities.

3.2 Synthesis of Workshop 2

In the second phase, a conceptual architecture oriented toward solving the problem through software was proposed. Approaches based on IoT devices or intensive centralized control were discarded due to institutional restrictions related to network, permissions, and deployment. As a result, a solution based on local agents operating in user space, interacting with operating system APIs, and reporting relevant events to external services was chosen.

3.3 Improvements Introduced in Workshop 3

In this stage, the design is refined from a robustness perspective. Fault tolerance mechanisms, quality assurance criteria, acceptance metrics, risk categorization, and a formal implementation structure are added. It is also recognized that a technical solution is insufficient without adequate project management, so schedule, resources, communications, control, and deployment preparation are incorporated.

3.4 Risks Identified and Mitigated

The main lesson is that an apparently simple solution can be strongly conditioned by the organizational and technical context. It was also learned that reliability in real environments requires considering contingencies from the design stage, especially when there are network blocks, variability in computational load, and sensitivity regarding potential loss of academic work. Finally, it became evident that documentation and traceability between phases strengthen project coherence and facilitate evaluation.

4 Robust System Architecture

4.1 Architecture Overview

The proposed architecture is decentralized and relies on an autonomous agent installed on each workstation participating in the pilot. This agent monitors user activity events, queries system status, and decides whether or not to apply a suspension or energy-saving policy. Generated records are ideally stored in a remote database; however, when connectivity is unavailable, a backup mechanism temporarily persists information locally.

4.2 Quality Attributes Mapping

System robustness is structured around several quality attributes. **Reliability** is supported by local persistence and deferred synchronization; **maintainability** is favored through modular component organization and separation between capture, decision, persistence, and communication; **scalability** is contemplated by allowing agents to be replicated across multiple equipment with minimal dependencies between them; **security** is addressed through encryption of local files and control of transmission paths; and **usability** is protected by avoiding suspensions when there are indications of relevant computational activity, even without direct mouse or keyboard interaction.

4.3 Integration and Interfaces

The system contemplates several main interfaces. First, a local interface with the operating system and monitoring libraries to identify activity events, peripheral status, and processor load. Second, a communication interface with external services, used to register events or send alerts. Third, an analytics interface to consume consolidated data and present it on a dashboard. These interfaces must operate under low coupling criteria, exception handling, and error traceability.

4.4 Fault-Tolerant Architecture

The robustified design proposes that the agent operates through independent modules or threads. If the communication channel with the cloud fails, the system must not stop or lose critical information; instead, it must locally store telemetry and suspension events in an encrypted CSV file. When network connectivity is restored, the system must synchronize pending data with the remote database. This reduces vulnerability to firewall blocks, temporary failures of external services, or communication channel interruptions.

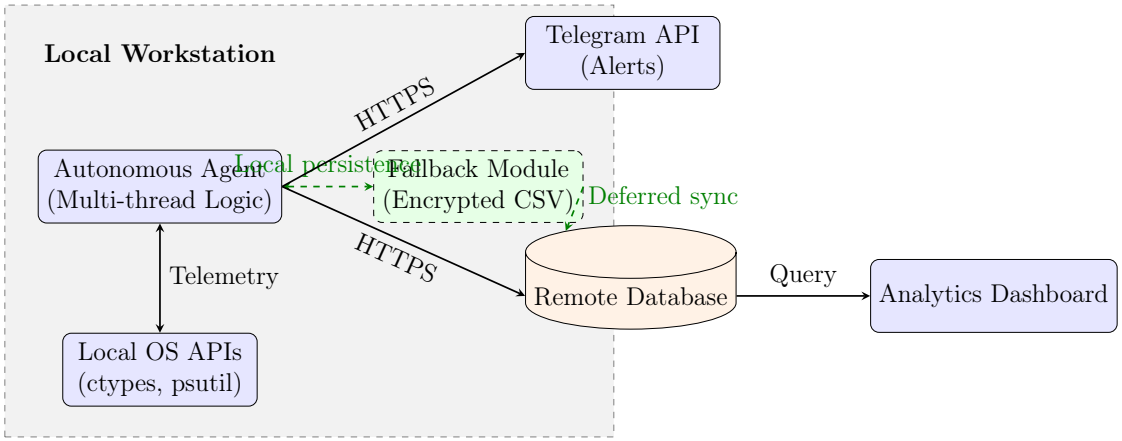

Figure 1: Decentralized robust architecture with fault tolerance and local persistence

5 Risk Management Plan

5.1 Risk Identification

A systematic risk identification was conducted considering technical, operational, security, and project management dimensions. The objective is to anticipate scenarios that could compromise system functionality, pilot continuity, or the quality of collected evidence.

5.2 Risk Categories

Technical risks include communication failures, agent logic errors, and unexpected operating system behavior. Operational risks encompass inadequate user usage, rejection of automation, or lack of timely access to laboratories. Security risks include exposure of sensitive telemetry information or unauthorized execution. Finally, project management risks comprise delays, excessive dependence on certain team members, limited time availability, and coordination difficulties.

5.3 Risk Matrix and Mitigation Strategies

Table 1: Project risk management matrix

ID	Risk	Prob.	Impact	Mitigation Strategy
R-01	HTTPS requests blocked by institutional firewall	High	Medium	Local encrypted CSV persistence and deferred synchronization when network becomes available.
R-02	False positive inactivity during heavy CPU tasks	Medium	Critical	CPU load verification before suspension; if load exceeds threshold, reset inactivity counter.
R-03	External alert service outage or unavailability	Low	Low	Structured exception handling and prioritization of local logic over external notifications.
R-04	Loss or corruption of local backup data	Low	High	Encryption, write validation, error handling, and periodic synchronization.
R-05	User resistance to automation	Medium	Medium	Prior communication, controlled pilot, explanation of purpose, and feedback collection.
R-06	Delays due to academic workload or dependency on few members	Medium	Medium	Clear responsibility distribution, weekly follow-up, and milestone-based progress review.

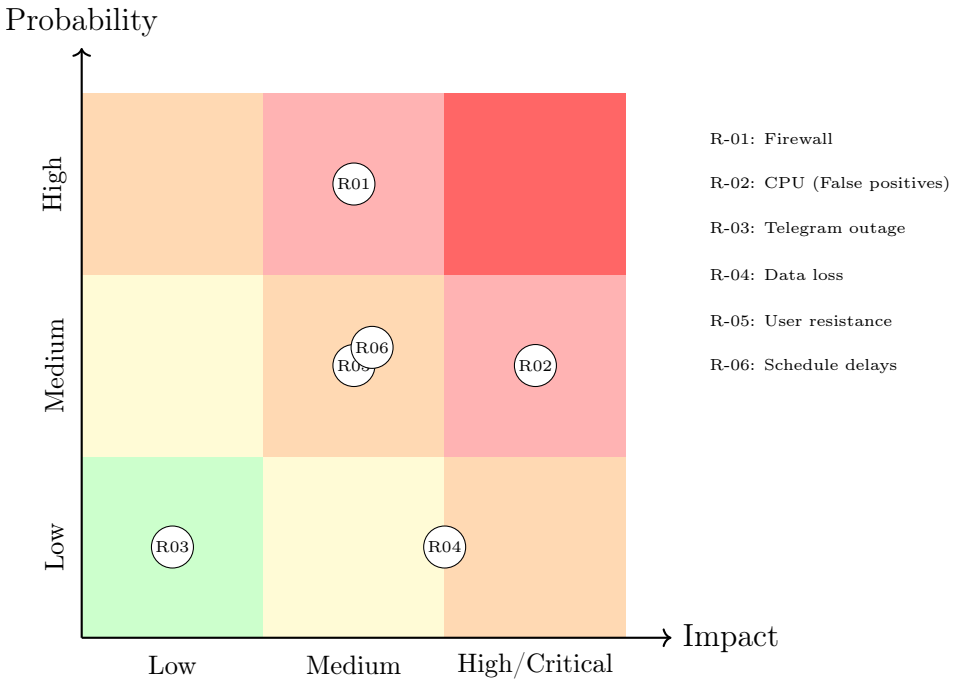

Figure 2: Matrix visualization of risk impact and probability

5.4 Risk Monitoring and Control

During the pilot, periodic monitoring of prioritized risks will be conducted. A weekly review of incident status, synchronization failures, agent behavior, and milestone compliance will be established. The quality and management lead will review collected evidence, classify events, and propose adjustments when recurrent error patterns are detected.

5.5 Contingency Plans

In case of network failures, local persistence will be activated; for erroneous inactivity detections, the decision threshold will be temporarily raised while parameters are adjusted; in case of user rejection, the pilot will be limited to clearly authorized equipment; and for schedule delays, critical deliverables will be prioritized: stable agent, test evidence, and results consolidation.

6 Quality Assurance Framework

6.1 Quality Objectives

The system must meet minimum objectives of functional correctness, operational reliability, fault recovery capability, maintainability, and suitability for the university context. Agent behavior must be predictable, auditable, and sufficiently flexible to adapt to different workloads without negatively affecting user experience.

6.2 Quality Metrics

The following metrics are proposed: rate of correct suspensions relative to total detected events; false positive inactivity rate; percentage of events stored without loss; average deferred data synchronization time; number of incidents per communication failure; and basic test coverage of critical modules. These metrics will allow measurement of both technical performance and operational stability.

6.3 Validation Methods

Validation will rely on unit tests, integration tests, computational stress tests, and controlled observation during the pilot. Unit tests will verify timing and inactivity logic; integration tests will evaluate persistence, reading, writing, and synchronization; stress tests will verify system response under heavy processes; and laboratory observations will identify discrepancies between expected design and real behavior.

6.4 Acceptance Criteria

Acceptance criteria will establish that the system must not suspend equipment during high computational load processes, maintain a high percentage of record retention even during network failures, exhibit reproducible behavior across all pilot equipment, and ensure the final dashboard receives consistent data for analysis. If minor failures occur, they will be evaluated to determine whether they compromise the main purpose of the Proof of Concept.

6.5 QA Flow

The quality assurance flow will follow a sequence of test case preparation, technical execution, findings registration, correction, revalidation, and approval for the next environment. This dynamic will ensure that the transition from alpha to beta phase is supported by evidence rather than mere perception of functionality.

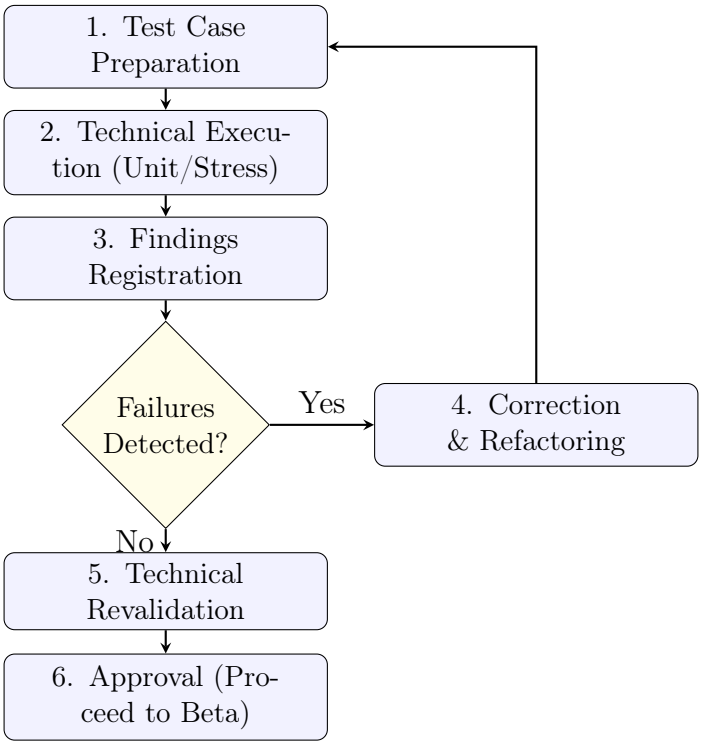

Figure 3: Iterative Quality Assurance process flow

7 Project Management Plan

7.1 Team Structure and Roles

Team organization responds to a functional distribution aimed at ensuring technical and operational coverage of the project.

Table 2: Team roles and responsibilities

Team Member	**Role**	**Key Responsibilities**
Omar Yesid Fonseca López	Project Manager & QA Lead	Schedule tracking, risk management, general coordination, and quality validation.
Daniel Felipe Barrera Suárez	Systems Architect	Robust architecture design, module integration, and structural decisions.
Daniel Mateo Ballesteros Molina	Algorithms & Agent Specialist	Agent implementation, monitoring logic, load validation, and local persistence.
Dania Lizeth Guzmán Triviño	Analytics & Communications	Data consolidation, dashboard, documentation support, and project communication strategy.

7.2 Project Schedule

A six-week execution period is proposed for the Proof of Concept, with defined milestones for development, testing, pilot, and consolidation.

Table 3: General project schedule

Activity	**W1**	**W2**	**W3**	**W4**	**W5**	**W6**
Architecture adjustment and robust design	X	X				
Agent and fallback implementation	X	X	X			
Internal testing and quality assurance			X	X		
Pilot deployment in laboratory				X	X	
Data consolidation and dashboard					X	X
Documentation closure and final evaluation						X

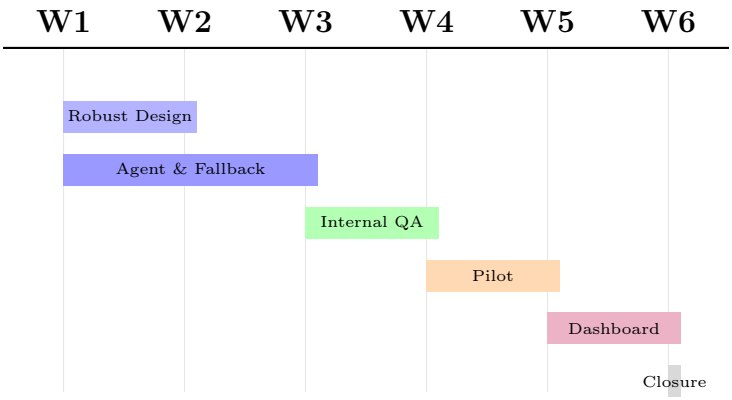

Figure 4: Visual timeline of the Proof of Concept (6 weeks)

7.3 Milestones, Deliverables, Dependencies, and Critical Path

The main milestones are: completion of robust design, implementation of stable agent, approval of internal tests, pilot deployment, dashboard consolidation, and documentation closure. Associated deliverables include agent code, risk matrix, QA results, telemetry records, and technical report. The critical path depends especially on agent stability and successful completion of the internal testing phase, as failure in this stage would directly impact pilot initiation.

7.4 Resource Plan

The project primarily requires human and software resources. No significant investment in additional hardware is planned since the solution runs on existing equipment. Python

and local monitoring libraries, CSV storage for contingency, a remote database for consolidation, and a visualization tool for the dashboard will be used. Team capacity is estimated at partial weekly dedication over six weeks, focused on development, testing, documentation, and results analysis.

7.5 Budget and Resource Plan

Given the software-defined nature of this Proof of Concept, no additional hardware procurement is required. The operational cost is based on the engineering team's time allocation (4 engineers × 20 hours/week × 6 weeks). All software tools used are open-source or have free tiers, making the PoC highly cost-effective and scalable for institutional adoption.

Table 4: Budget estimation for the PoC execution

Resource	**Type**	**Cost**	**Notes**
Engineering team (4×20h×6 weeks)	Human	$0 (academic)	Course project effort
Cloud database (MySQL DBaaS)	Software	$0	Free tier (e.g., Clever Cloud, Aiven)
Telegram Bot API	Software	$0	Free for PoC scale
Power BI / Looker Studio	Software	$0	Free tier available
Python libraries (psutil, requests, cryptography)	Software	$0	Open source
GitHub repository	Infrastructure	$0	Student pack / free
Total estimated cost		**$0 USD (academic PoC)**	

7.6 Communication and Control Plan

Weekly internal meetings will be established to review progress, risks, and blockers. Each team member will report the status of their tasks and relevant findings. For external stakeholders, especially laboratory faculty or supervisors, brief communications will be prepared prior to the pilot indicating purpose, scope, involved equipment, and precautionary measures. Project control will be carried out through milestone tracking, deliverable review, and periodic validation of risk and quality status.

8 Implementation and Readiness Strategy

8.1 Deployment Strategy

Implementation will follow a gradual approach. First, an alpha phase will be executed on the team's own equipment to validate basic agent behavior. Then, a beta phase will

proceed on a small set of laboratory computers with prior authorization. Finally, data consolidation and pilot results analysis will be performed.

Table 5: Pilot implementation phases

Phase	Scope	Main Activities	Acceptance Criteria
Alpha	4 team member laptops	Logic verification, internal testing, fallback validation, log review.	Stable operation without improper suspensions.
Beta	5 laboratory computers	Controlled deployment, real observation, telemetry collection, parameter adjustment.	Useful evidence and consistent behavior in real environment.
Closure	Analytical consolidation	Synchronization, dashboard, preliminary savings and event analysis.	Integrated data ready for final evaluation.

8.2 Infrastructure Requirements

Access to authorized laboratory equipment, local execution permissions for the agent, eventual connectivity for data synchronization, and a remote database environment or central repository to store consolidated telemetry are required. A visualization tool is also needed to build the final pilot dashboard.

8.3 Dependencies and Preconditions

Pilot execution depends on authorization to intervene in a controlled set of equipment, availability of team members to conduct tests and monitoring, and correct agent configuration on each station. A procedure to extract or synchronize data in case of limited connectivity during the test is also necessary.

8.4 Change Management

Any modification to suspension logic, decision thresholds, or persistence format must be registered and justified. The goal is to avoid improvised changes during the pilot. Adjustments must be approved by the component lead and communicated to the rest of the team to maintain traceability and coherence between documentation, code, and results.

8.5 Training Plan

Before pilot deployment, team members must review the installation, execution, monitoring, and failure recovery procedures. Additionally, a brief explanation will be prepared for laboratory supervisors or involved users, emphasizing the pilot's purpose, expected effects, and channels for reporting incidents or concerns.

8.6 User Adoption Strategy

Although the pilot has limited scope, it is important to foster a positive perception of the solution. It will be explained that the system seeks to optimize resources without interfering with legitimate academic work. An approach of accompaniment and observation will be privileged over strict imposition, collecting feedback for future iterations.

9 Continuous Improvement Strategy

9.1 Feedback Mechanisms

Continuous improvement will rely on three main sources: technical test results, pilot observations, and comments from users or laboratory supervisors. Each of these sources will allow detection of necessary adjustments in thresholds, wait times, record formats, and system behavior in specific contexts.

9.2 Adaptive Management

The project assumes that some parameters will need calibration based on usage experience. Therefore, the architecture and documentation have been designed in a modular and adjustable manner, allowing improvements to be introduced without completely redesigning the solution. This adaptive capacity is especially valuable in a real environment with changing restrictions.

9.3 Post-Implementation Metrics

After the pilot, metrics such as number of suspension events, percentage of valid events, reported incidents, volume of data recovered through deferred synchronization, and basic perception of system utility will be evaluated. These metrics will allow assessment not only of whether the system works, but also whether it generates sufficient evidence to justify a subsequent expansion phase.

10 Visual Representations and Tools

This document incorporates diagrams and tables that facilitate understanding of the architecture, risks, and project planning. For an extended version of the work, it is recommended to complement with a formal Gantt chart, a matrix visualization of risk impact and probability, and a specific flow of the quality assurance process. Tools such as draw.io, Lucidchart, Microsoft Project, or TikZ can be used to improve visual presentation and project traceability.

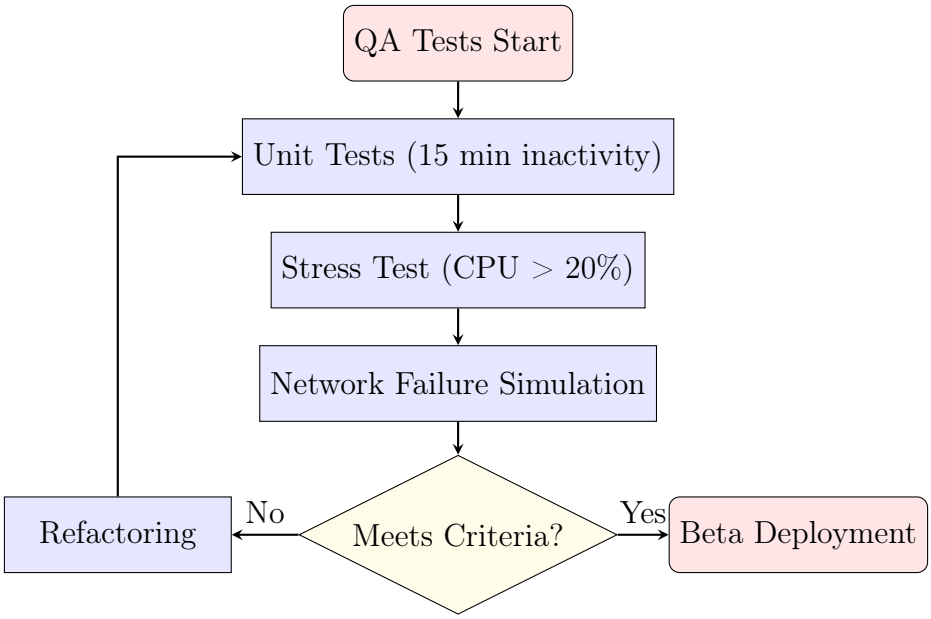

Figure 5: Quality Assurance and Testing Framework Flow

11 References

1. International Organization for Standardization. *ISO 31000: Risk Management – Guidelines*. ISO, 2018.

2. International Organization for Standardization and International Electrotechnical Commission. *ISO/IEC 25010: Systems and Software Quality Requirements and Evaluation (SQuaRE) – System and Software Quality Models*. ISO/IEC, 2011.

3. Project Management Institute. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 7th ed., PMI, 2021.

4. Carnegie Mellon University. *CMMI for Development, Version 2.0*. CMMI Institute, 2018.

5. Institute of Electrical and Electronics Engineers. *IEEE Standard for Software Reviews and Audits*. IEEE.

6. T. Hong, H. Taylor-Lange, S. D'Oca, D. Yan, and S. Corgnati, "Advances in research and applications of energy-related occupant behavior in buildings," *Energy and Buildings*, vol. 116, pp. 694-702, 2016.

7. C. A. Sierra, "Workshop No. 1: Systems Analysis," Systems Analysis & Design Course, Universidad Distrital Francisco José de Caldas, 2026.

12 Appendices

12.1 Repository Structure and Deliverables

The following files and folders are available in the project GitHub repository (https://github.com/yecop/Systems-Analysis-and-Design):

Table 6: Repository structure and workshop deliverables

Folder / File	Workshop	Description
/docs/Workshop1_Analysis.pdf	1	Final IEEE-formatted systems analysis document
/data/observacion_directa.csv	1	Direct observation logs (occupancy vs. energy waste)
/data/luminosidad_*.csv	1	Environmental lux meter correlation data (3 cases)
/data/Datos_encuestas.csv	1	Raw structured survey responses (n=35)
/assets/diagrams/	1,2,3	Architecture diagrams and visual representations
/docs/Workshop2_Design.pdf	2	Software-defined energy management PoC design
/src/agent/	2,3	Autonomous telemetry agent source code (Python)
/Workshop3_Management/	3	This document (PDF) and supporting management artifacts
/README.md	1,2,3	Project overview and navigation (updated)

12.2 Quality Assurance Checklist

The following checklist was used during the QA phase (referenced in Section 6):

- Unit test: Idle detection accuracy (target: 100% after 15 min of inactivity)
- Unit test: CPU load verification before suspension (threshold: <20%)
- Integration test: Local CSV encryption and write validation
- Integration test: Fallback activation on network failure simulation
- Stress test: Video rendering benchmark (no false suspension during >20% CPU)
- Network test: Deferred synchronization after connectivity restoration

12.3 Detailed Risk Register

Complete risk register extending Table 1:

ID	Risk	Prob.	Impact	Trigger	Contingency
R-01	Firewall blocks HTTPS requests	High	Medium	HTTP 403/Timeout	Local CSV fallback
R-02	False positive on high CPU load	Medium	Critical	CPU >20% for >5 min	Reset counter, log event
R-03	Telegram API outage	Low	Low	API timeout	Skip notification, suspend anyway
R-04	Local data corruption	Low	High	CSV checksum fails	Periodic backup, retry sync
R-05	User resistance	Medium	Medium	User complaints	Pilot on authorized PCs only
R-06	Schedule delays	Medium	Medium	Missed milestone	Prioritize critical path

Table 7: Detailed risk register

12.4 Project Repository Link

The complete project repository can be accessed at: https://github.com/yecop/Systems-Analysis-a

All documentation, source code, datasets, and diagrams are available under an academic use license.